# React Fundamentals

A beginner-friendly guide to building user interfaces with React

Covers: setup • JSX • components & props • state • events • conditional rendering • lists & keys • forms • useEffect • other hooks • custom hooks • routing • data fetching • best practices

# Contents

1. Introduction to React
2. Setting Up a Project
3. JSX
4. Components and Props
5. State with useState
6. Handling Events
7. Conditional Rendering
8. Lists and Keys
9. Forms
10. Side Effects with useEffect
11. Other Essential Hooks
12. Custom Hooks
13. Routing with React Router
14. Fetching Data
15. Best Practices and Checklist

# 1. Introduction to React

**React** is a JavaScript library for building user interfaces, created at Facebook (now Meta) and released in 2013. You describe what the UI should look like for a given state, and React updates the browser DOM efficiently when that state changes.

## Core ideas

| Idea | Meaning |
|---|---|
| Component-based | UIs are built from small, reusable, self-contained pieces. |
| Declarative | You describe the desired UI; React figures out how to update the DOM. |
| Unidirectional data flow | Data flows down from parent to child through props. |
| Virtual DOM & reconciliation | React compares the new UI description with the previous one and applies only the minimal DOM changes. |
| Hooks | Functions such as useState and useEffect that add state and lifecycle behaviour to function components. |

**Prerequisites:** solid HTML and CSS, and modern JavaScript (ES6+): arrow functions, destructuring, spread syntax, template literals, modules, and array methods like map and filter.

# 2. Setting Up a Project

The recommended way to start today is **Vite**, a fast build tool. You need Node.js installed.

```
npm create vite@latest my-app -- --template react
cd my-app
npm install
npm run dev
```

Open the URL shown in the terminal (usually http://localhost:5173).

```
my-app/
  index.html
  package.json
  src/
    main.jsx        <- entry point, mounts the app
    App.jsx         <- root component
    components/     <- reusable components

// src/main.jsx
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

# 3. JSX

JSX is a syntax extension that lets you write HTML-like markup inside JavaScript. Build tools compile it to regular JavaScript calls.

```
const name = "Asha";
const element = (
  <div className="card">
```

```
    <h1>Hello, {name}!</h1>
    <p>2 + 2 = {2 + 2}</p>
    <img src="/logo.png" alt="Logo" />
  </div>
);
```

### JSX rules

- Return a **single root element**; use a fragment `<>...</>` to avoid an extra wrapper div.
- Close every tag, including self-closing ones: `<img />`, `<input />`.
- Use `className` instead of `class`, and `htmlFor` instead of `for`.
- Use camelCase for attributes and events: `onClick`, `tabIndex`.
- Embed JavaScript *expressions* in curly braces `{ }`; statements like if or for are not allowed inside.
- Inline styles take an object: `style={{ color: "red", fontSize: 16 }}`.

## 4. Components and Props

A component is a JavaScript function that returns JSX. Component names must start with a **capital letter**.

```
function Greeting({ name, age = 18 }) {
  return <p>Hello {name}, you are {age}.</p>;
}

export default function App() {
  return (
    <>
      <Greeting name="Asha" age={25} />
      <Greeting name="Ravi" />
    </>
  );
}
```

### Props

- Props are inputs passed from parent to child, like function arguments.
- Props are **read-only**; a component must never modify its own props.
- Any JavaScript value can be a prop: strings, numbers, objects, functions, even other components.
- The special `children` prop holds whatever is nested between a component's tags.

```
function Card({ title, children }) {
  return (
    <div className="card">
      <h2>{title}</h2>
      {children}
    </div>
  );
}

<Card title="Welcome"><p>This is the card body.</p></Card>
```

## 5. State with useState

State is data that changes over time and, when updated, causes the component to re-render. Use the `useState` hook.

```
import { useState } from "react";
```

```
function Counter() {
  const [count, setCount] = useState(0);   // [value, setter]

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increment</button>
      <button onClick={() => setCount(c => c - 1)}>Decrement</button>
    </div>
  );
}
```

### Important rules about state

- Never mutate state directly; always call the setter (`count = 5` does nothing useful).
- Use the **updater form** `setCount(c => c + 1)` when the new value depends on the previous one.
- State updates are batched and applied on the next render.
- For objects and arrays, create a **new copy** instead of mutating the existing one.
- Keep state minimal; derive values from state during rendering instead of storing them.

```
const [user, setUser] = useState({ name: "Asha", age: 25 });
setUser({ ...user, age: 26 });                // update object

const [items, setItems] = useState([1, 2, 3]);
setItems([...items, 4]);                      // add
setItems(items.filter(i => i !== 2));         // remove
setItems(items.map(i => i === 3 ? 30 : i));   // update
```

# 6. Handling Events

Pass a **function** to event props such as `onClick`, `onChange` and `onSubmit`. Do not call it: write `onClick={handle}`, not `onClick={handle()}`.

```
function Search() {
  const [text, setText] = useState("");

  function handleChange(event) {
    setText(event.target.value);
  }

  return (
    <input value={text} onChange={handleChange} placeholder="Search..." />
  );
}
```

Use `event.preventDefault()` to stop default browser behaviour (for example, a form submit reloading the page).

# 7. Conditional Rendering

```
function Status({ isLoggedIn, unread }) {
  if (!isLoggedIn) return <p>Please log in.</p>;       // early return

  return (
    <div>
      <h2>Welcome back!</h2>
      {unread > 0 && <span>{unread} new messages</span>}   {/* && */}
      <p>{unread === 0 ? "Inbox empty" : "Check your inbox"}</p> {/* ternary */}
    </div>
  );
}
```

Be careful with && and numbers: `{count && <X />}` renders a literal **0** when count is 0. Use `count > 0 && ...` instead.

# 8. Lists and Keys

Render lists with `map()`. Every item needs a stable, unique `key` so React can track items between renders.

```
const todos = [
  { id: 1, text: "Learn React", done: true },
  { id: 2, text: "Build a project", done: false },
];

function TodoList() {
  return (
    <ul>
      {todos.map(todo => (
        <li key={todo.id} style={{ textDecoration: todo.done ? "line-through" : "none" }}>
          {todo.text}
        </li>
      ))}
    </ul>
  );
}
```

- Use a database id or other unique value as the key.
- Avoid using the array index as a key if the list can be reordered, filtered or have items inserted.
- Keys only need to be unique among siblings, and are not passed as a prop.

# 9. Forms

In a **controlled component**, form values live in React state and the input displays that state, making React the single source of truth.

```
function SignupForm() {
  const [form, setForm] = useState({ name: "", email: "", agree: false });

  function handleChange(e) {
    const { name, value, type, checked } = e.target;
    setForm({ ...form, [name]: type === "checkbox" ? checked : value });
  }

  function handleSubmit(e) {
    e.preventDefault();
    console.log("Submitting", form);
  }

  return (
    <form onSubmit={handleSubmit}>
      <input name="name" value={form.name} onChange={handleChange} />
      <input name="email" type="email" value={form.email} onChange={handleChange} />
      <label>
        <input name="agree" type="checkbox" checked={form.agree} onChange={handleChange} />
        I agree
      </label>
      <button type="submit" disabled={!form.agree}>Sign up</button>
    </form>
  );
}
```

For larger forms, libraries such as **React Hook Form** reduce boilerplate and re-renders, and **Zod** can validate the data.

# 10. Side Effects with useEffect

A side effect is anything that reaches outside rendering: fetching data, subscriptions, timers, or manually changing the document. Run them with `useEffect`.

```
import { useState, useEffect } from "react";

function Clock() {
  const [time, setTime] = useState(new Date());

  useEffect(() => {
    const id = setInterval(() => setTime(new Date()), 1000);
    return () => clearInterval(id);        // cleanup on unmount
  }, []);                                  // dependency array

  return <p>{time.toLocaleTimeString()}</p>;
}
```

| Dependency array | When the effect runs |
|---|---|
| `useEffect(fn)` | After every render |
| `useEffect(fn, [])` | Once, after the first render |
| `useEffect(fn, [a, b])` | After the first render and whenever a or b changes |

- List **every** reactive value used inside the effect in the dependency array.
- Return a cleanup function to remove listeners, timers and subscriptions.
- In development, StrictMode runs effects twice to help you find missing cleanup.
- You may not need an effect: calculate derived values during render and handle user actions in event handlers.

# 11. Other Essential Hooks

| Hook | Purpose |
|---|---|
| `useRef` | Holds a mutable value that does not trigger re-render; also accesses DOM nodes. |
| `useContext` | Reads a value from a context, avoiding prop drilling. |
| `useReducer` | Manages complex state transitions with a reducer function. |
| `useMemo` | Caches the result of an expensive calculation between renders. |
| `useCallback` | Caches a function definition between renders. |

## useRef

```
function FocusInput() {
  const inputRef = useRef(null);
  return (
    <>
      <input ref={inputRef} />
      <button onClick={() => inputRef.current.focus()}>Focus</button>
    </>
  );
}
```

### useContext

```
const ThemeContext = createContext("light");

function App() {
  return (
    <ThemeContext.Provider value="dark">
      <Toolbar />
    </ThemeContext.Provider>
  );
}

function Toolbar() {
  const theme = useContext(ThemeContext);
  return <div className={theme}>Current theme: {theme}</div>;
}
```

### useReducer

```
function reducer(state, action) {
  switch (action.type) {
    case "add":    return [...state, action.text];
    case "remove": return state.filter((_, i) => i !== action.index);
    default:       return state;
  }
}

const [todos, dispatch] = useReducer(reducer, []);
dispatch({ type: "add", text: "Learn hooks" });
```

**Rules of Hooks:** call hooks only at the top level of a component or custom hook (never inside loops, conditions or nested functions), and only from React functions.

## 12. Custom Hooks

A custom hook is a function whose name starts with `use` that calls other hooks. It lets you reuse stateful logic across components.

```
function useLocalStorage(key, initialValue) {
  const [value, setValue] = useState(() => {
    const saved = localStorage.getItem(key);
    return saved ? JSON.parse(saved) : initialValue;
  });

  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);

  return [value, setValue];
}

// usage
const [theme, setTheme] = useLocalStorage("theme", "light");
```

## 13. Routing with React Router

React is only a UI library, so client-side routing comes from a library such as **React Router**.

```
npm install react-router-dom

import { BrowserRouter, Routes, Route, Link, useParams } from "react-router-dom";

function UserPage() {
  const { id } = useParams();
  return <h2>User #{id}</h2>;
}
```

```
export default function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link> | <Link to="/about">About</Link>
      </nav>
      <Routes>
        <Route path="/" element={<h1>Home</h1>} />
        <Route path="/about" element={<h1>About</h1>} />
        <Route path="/users/:id" element={<UserPage />} />
        <Route path="*" element={<h1>404 Not Found</h1>} />
      </Routes>
    </BrowserRouter>
  );
}
```

Use `Link` or `NavLink` instead of plain anchor tags to navigate without a full page reload.

# 14. Fetching Data

```
function Users() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const controller = new AbortController();

    fetch("https://jsonplaceholder.typicode.com/users", { signal: controller.signal })
      .then(res => {
        if (!res.ok) throw new Error("Request failed: " + res.status);
        return res.json();
      })
      .then(data => setUsers(data))
      .catch(err => { if (err.name !== "AbortError") setError(err.message); })
      .finally(() => setLoading(false));

    return () => controller.abort();       // cancel if unmounted
  }, []);

  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error}</p>;
  return <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}
```

For production apps, consider **TanStack Query** (React Query) or **SWR**. They handle caching, retries, background refetching and loading/error states for you.

# 15. Best Practices and Checklist

## Common patterns

- **Lifting state up:** when two components need the same data, move the state to their closest common parent and pass it down.
- **Composition over inheritance:** build complex UIs by combining components and using children.
- **Single responsibility:** keep components small and focused.
- **Colocate state** as close as possible to where it is used.
- Use **React.memo**, useMemo and useCallback only after measuring a real performance problem.

## Checklist

- [ ] Component names start with a capital letter; one component per responsibility
- [ ] State is never mutated directly; arrays and objects are copied before updating
- [ ] Every list item has a stable, unique key
- [ ] Effects have correct dependency arrays and cleanup functions
- [ ] Forms use controlled inputs and validate user input
- [ ] Hooks follow the Rules of Hooks
- [ ] Loading and error states are handled for all async operations
- [ ] Accessibility basics: semantic HTML, alt text, labels, keyboard support
- [ ] Code is linted (ESLint) and formatted (Prettier)
- [ ] Components and hooks are tested with Vitest/Jest and React Testing Library

## Suggested next steps

- Build projects: a to-do app, a weather app using a public API, and a small shop with a cart.
- Learn TypeScript with React, and a state library such as Zustand or Redux Toolkit.
- Explore frameworks built on React such as Next.js, and read the official docs at react.dev.